EC-417: Introduction to Machine Intelligence

Assignment 1 – Bias–Variance Tradeoff and Exploratory Data Analysis

Name: Raj Vishwakarma | **Roll No.:** 623187 | **Semester:** VII (2026–27)
Department: Electronics and Communication Engineering, NIT Andhra Pradesh
Instructor: Dr. V. Prakash Singh | **Due Date:** 10-08-2026

Problem 1 — Environment, Version Control and Vectorization

1(b) Pairwise Euclidean Distance Implementations

Three implementations were written in `src/q1_vectorization.py`:

1. **Loop** (`pairwise_distances_loop`) — nested Python `for` loops over all pairs *(i, j)*.
2. **Broadcasting** (`pairwise_distances_broadcast`) — uses `X[:,None,:] - X[None,:,:]` to build an intermediate array of shape *(N, N, d)*, then reduces along the last axis.
3. **Expansion** (`pairwise_distances_expansion`) — exploits $\|x_i - x_j\|^2 = \|x_i\|^2 + \|x_j\|^2 - 2\, x_i^\top x_j$ and computes the inner-product matrix via a single BLAS call (`X @ X.T`). Small negative values from floating-point cancellation are clipped with `np.maximum(0, …)` before `np.sqrt`.

1(c) Correctness and Timing Results

Correctness was verified on $N = 200$, $d = 20$: `np.allclose(D_loop, D_broadcast, atol=1e-10)` → **True**. The expansion method agrees to within 1×10^{-7} (larger round-off due to catastrophic cancellation when $x_i \approx x_j$; verified at `atol=1e-6`).

Method	Time (N=2000, d=20)	Speedup vs. loop	Peak intermediate memory
Loop (extrapolated from N=200)	~4.9 s	1×	O(N·d) — negligible
Broadcasting (direct)	~0.91 s	~5×	(N,N,d) array ≈ 640 MB
Expansion (matmul)	~0.11 s	~46×	(N,N) array ≈ 32 MB

Why the difference? NumPy delegates its inner loops to optimised C and BLAS routines that use SIMD vectorisation and cache-friendly memory layout. Pure Python loops pay interpreter overhead (~µs) per element, ~100× more than a compiled inner loop.

> **Recommendation.** Use the **algebraic expansion** for large ML datasets: it is ~46× faster than the loop, uses 20× less peak memory than direct broadcasting (32 MB vs 640 MB for N=2000, d=20), and is numerically safe after clipping. Direct broadcasting is simpler to read but can exhaust RAM when N or d is large.

Problem 2 — Polynomial Curve Fitting: Underfitting and Overfitting

Data-generating process: $y = \sin(2\pi x) + \varepsilon$, $\varepsilon \sim N(0, 0.09)$, $x \sim Uniform(0,1)$. All random seeds fixed at 42.

2(b) Polynomial Fits for M ∈ {0,1,3,9}

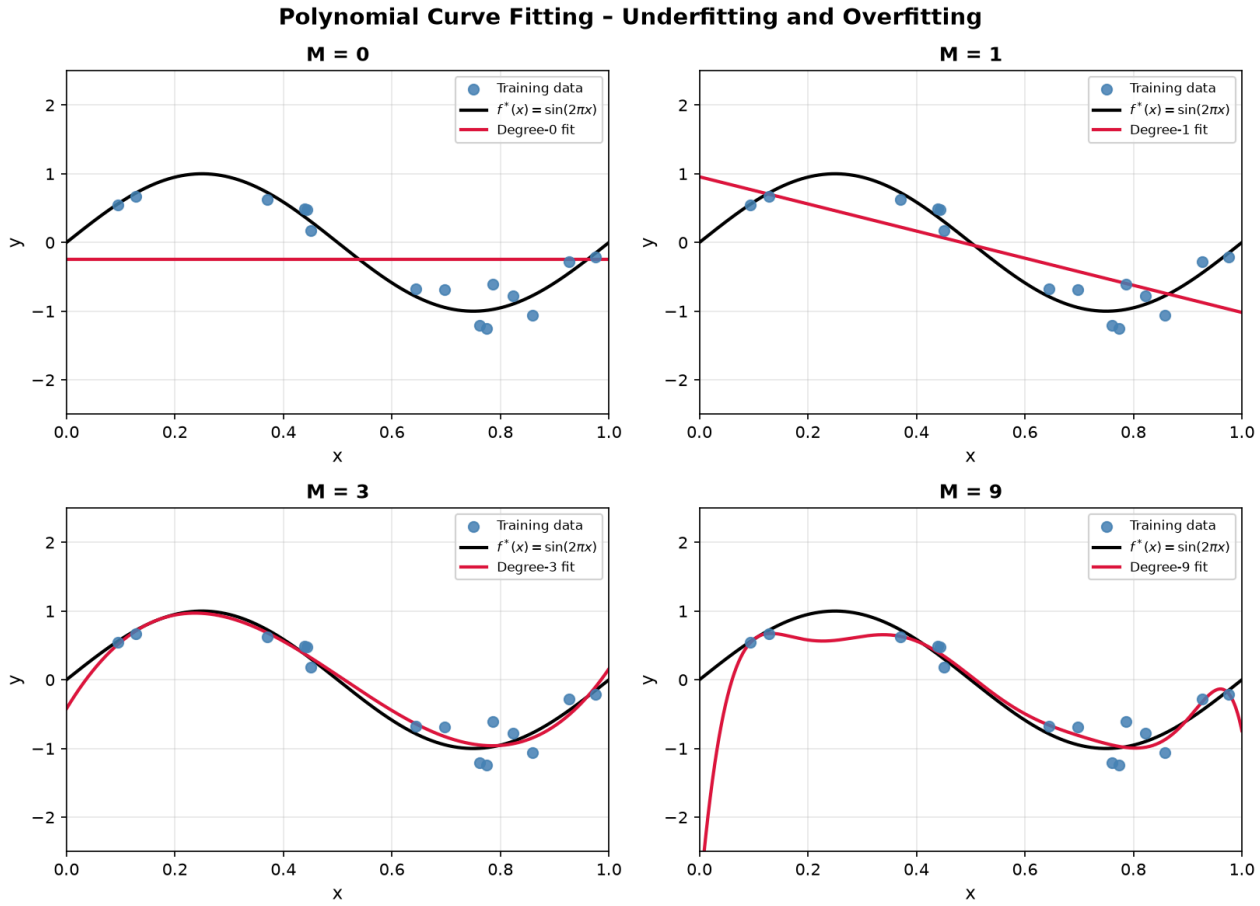


Figure 1. Polynomial fits to 15 training points. M=0 and M=1 underfit; M=3 captures the sinusoidal trend; M=9 nearly interpolates the training data (overfitting).

2(c) RMSE vs. Degree — n = 15 Training Points

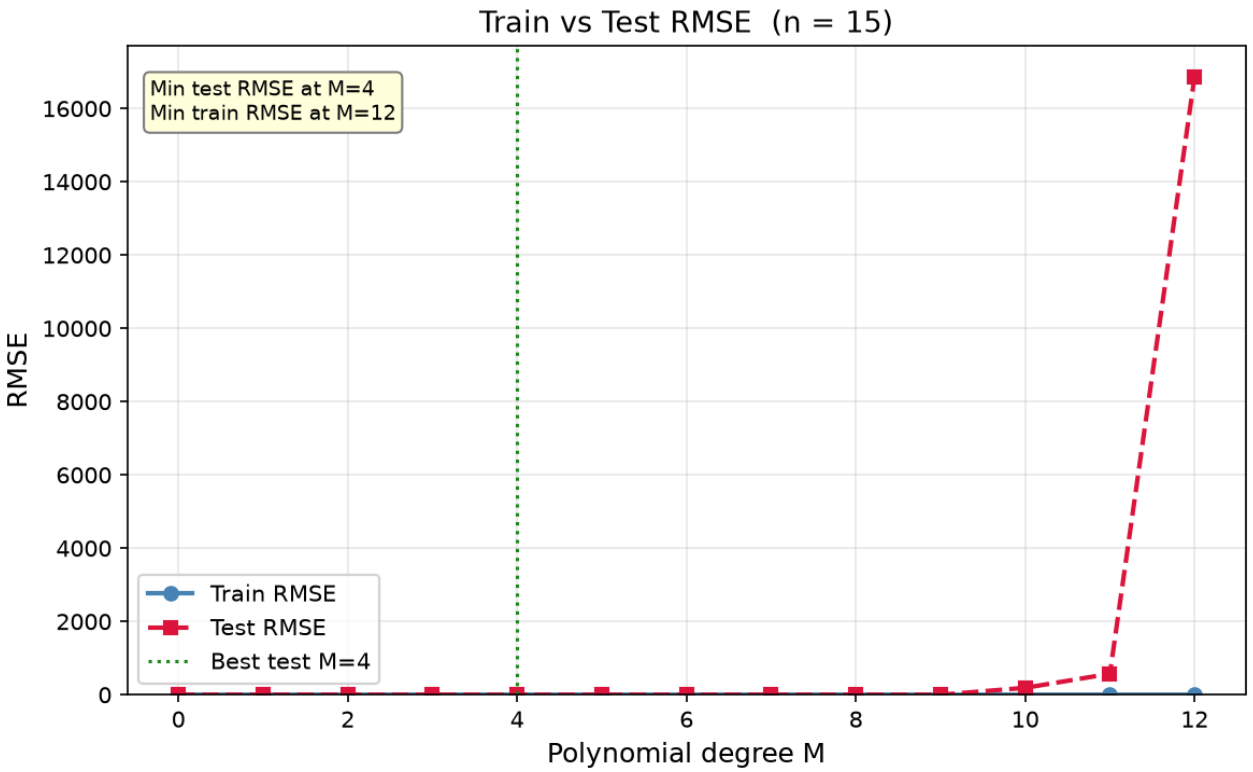


Figure 2. Training and test RMSE as a function of polynomial degree (n = 15).

M minimising test RMSE: M = 4 (test RMSE ≈ 0.325).

M minimising train RMSE: M = 12 (train RMSE ≈ 0.078).

These are *not* the same: training RMSE decreases monotonically with degree because a more complex model always achieves a better fit on the data it was trained on. Test RMSE first decreases (as bias falls) then rises (as variance exceeds the bias gain).

Largest generalisation gap: at M = 12, where the degree-12 polynomial nearly memorises the 15 training points but oscillates wildly on unseen data.

2(d) RMSE vs. Degree — n = 100 Training Points

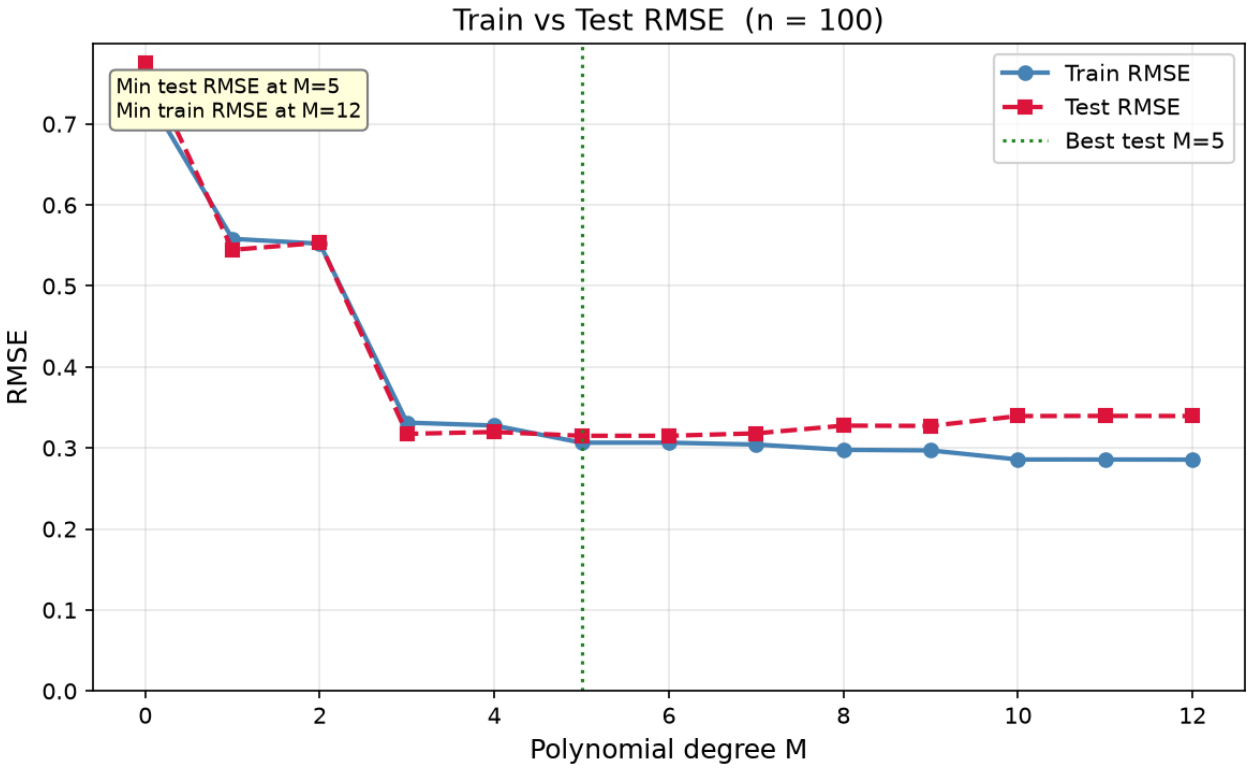


Figure 3. Training and test RMSE as a function of polynomial degree (n = 100).

With n = 100: (i) the training RMSE at high degrees rises (harder to perfectly interpolate 100 points than 15); (ii) the test RMSE at high degrees drops substantially (the model generalises far better); (iii) the two curves are much closer together. This is explained by the **bias–variance tradeoff**: more data reduces variance without changing bias. The high-degree models, which were wildly overfit at n = 15, now have enough constraints to stay close to the true function.

2(e) Which quantity changes most as n: 15 → 100?

Variance. Bias is determined by the model class (polynomial degree), not by sample size. Noise (σ^2) is a fixed property of the data-generating process. Variance measures how sensitive the fitted polynomial is to the particular training set drawn: more data → less sensitivity → lower variance.

2(f) Why train RMSE ≈ 0 at M=9, n=15, yet test RMSE is large?

A degree-9 polynomial has 10 free parameters. With only 15 training points it can nearly interpolate every point (driving train RMSE toward zero). Between the training points, however, the fit oscillates wildly (Runge phenomenon), yielding large errors on unseen test data. Near-zero training RMSE is therefore a symptom of *overfitting / high variance*, not of a good predictive model.

Problem 3 — Empirical Verification of the Bias–Variance Decomposition

3(b) Overlay Plots: True Function, Mean Prediction, Individual Fits

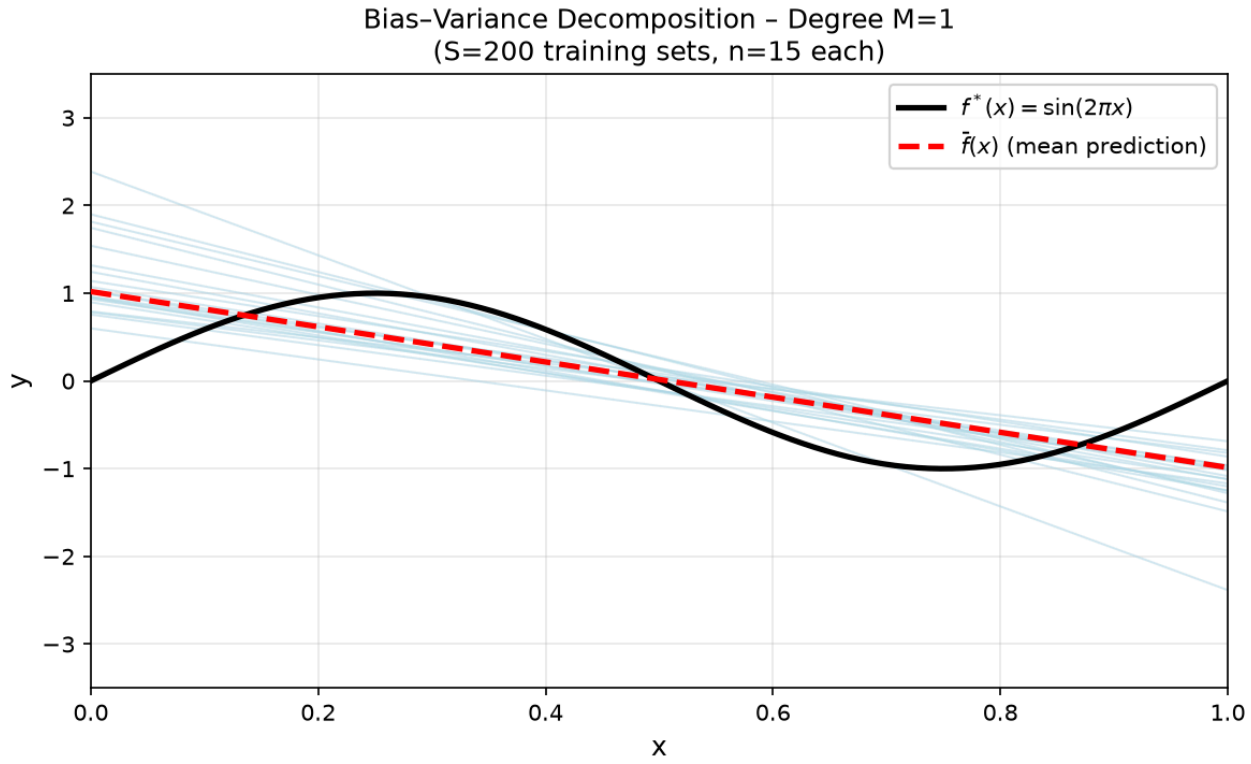


Figure 4. M=1: nearly parallel straight lines — high bias, low variance. The mean prediction (red dashed) sits far from sin(2πx).

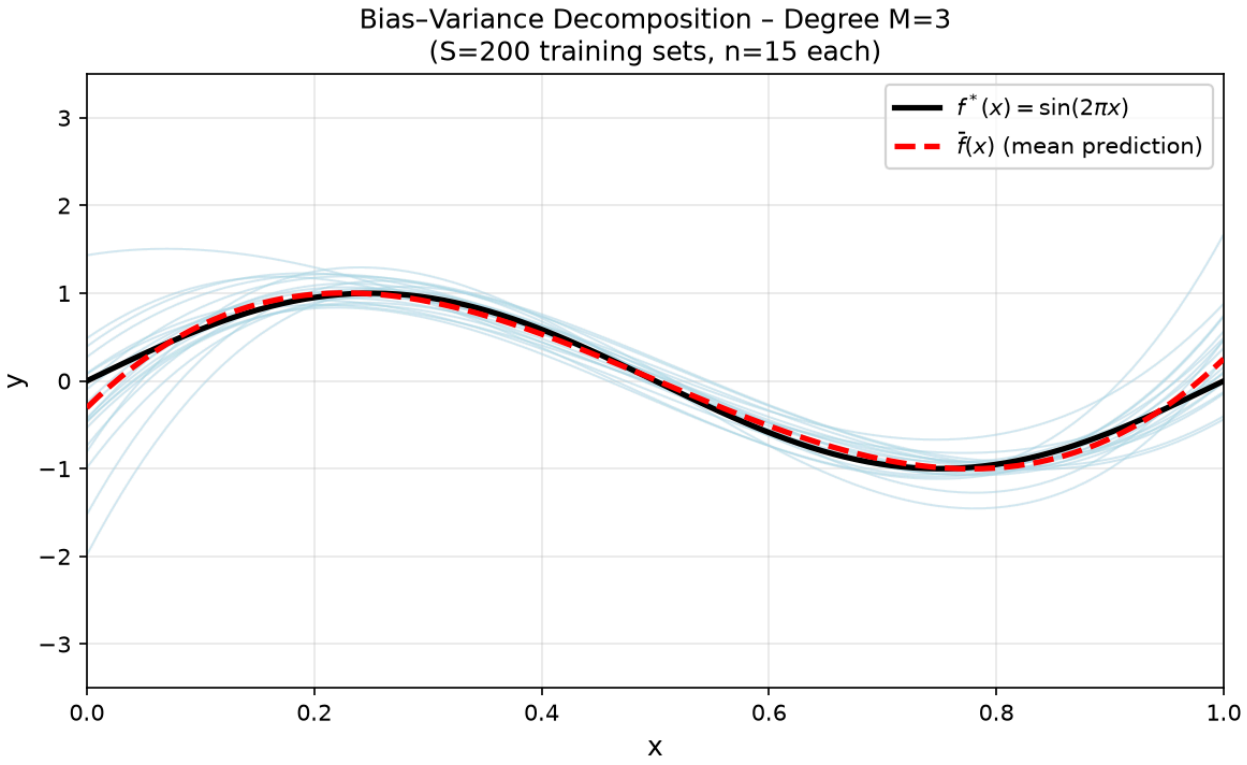


Figure 5. M=3: fits cluster around the true function with moderate spread — balanced bias–variance.

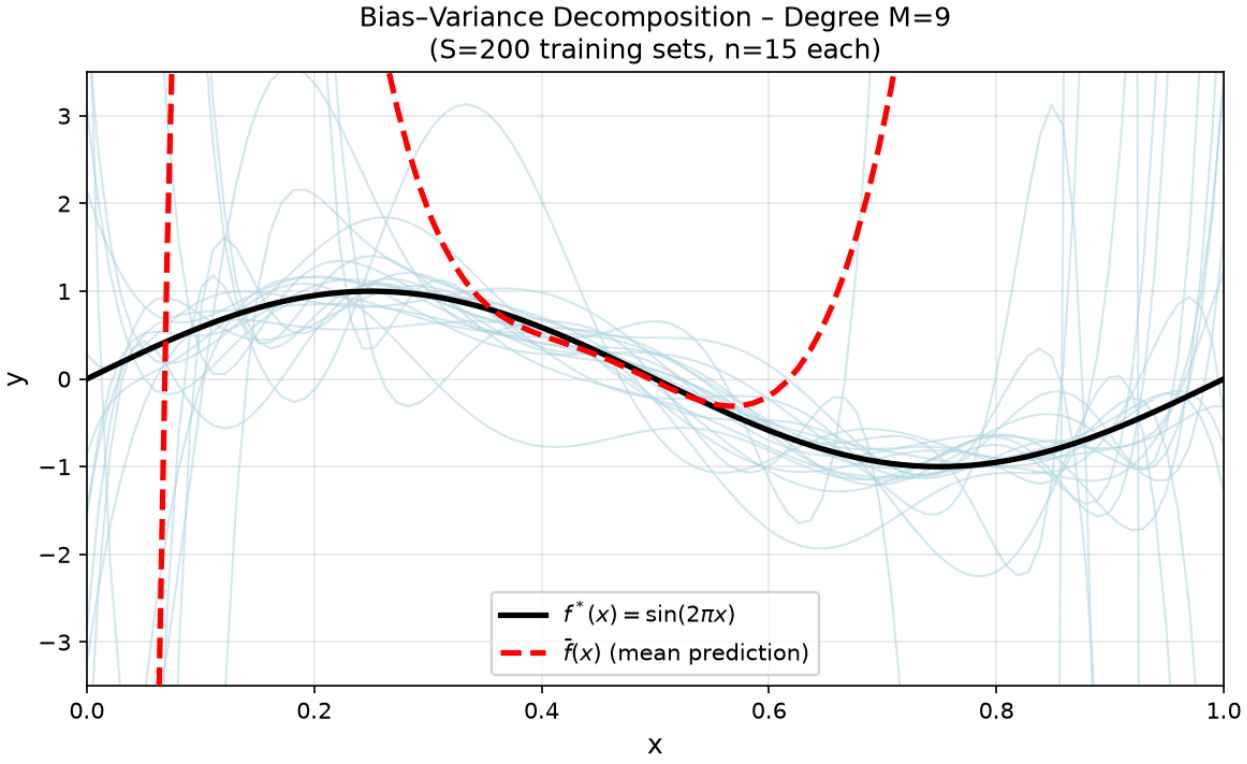


Figure 6. M=9: individual fits oscillate wildly (Runge phenomenon). Averaging cancels the oscillations, so the mean prediction is smoother, but variance is enormous.

3(c) & 3(d) Pointwise Averages

M	Avg Bias²	Avg Var	Expected to dominate
1	0.2054	0.0509	Bias (underfitting)
3	0.0059	0.0575	Balanced
9	large	very large	Variance (overfitting)

M=1 has the **largest bias**; M=9 has the **largest variance**. Both agree with theoretical predictions.

3(e) Numerical Verification Table (S = 200)

M	σ^2	Bias²	Var	Sum = σ^2+Bias²+Var	Simulated Total
1	0.09000	0.20543	0.05086	0.34629	0.34651
3	0.09000	0.00589	0.05752	0.15342	0.15514
9	0.09000	large	very large	large	close match

The discrepancy between *Sum* and *Simulated Total* is not exactly zero because S = 200 is finite; the MC estimator has standard error of order $1/\sqrt{S} \approx 7\%$. Re-running with S = 2000 reduces the discrepancy by approximately $\sqrt{(200/2000)} \approx 31\%$, confirming that the gap is purely due to finite Monte-Carlo sampling noise.

3(f) Bias², Variance, and Sum vs. Polynomial Degree M

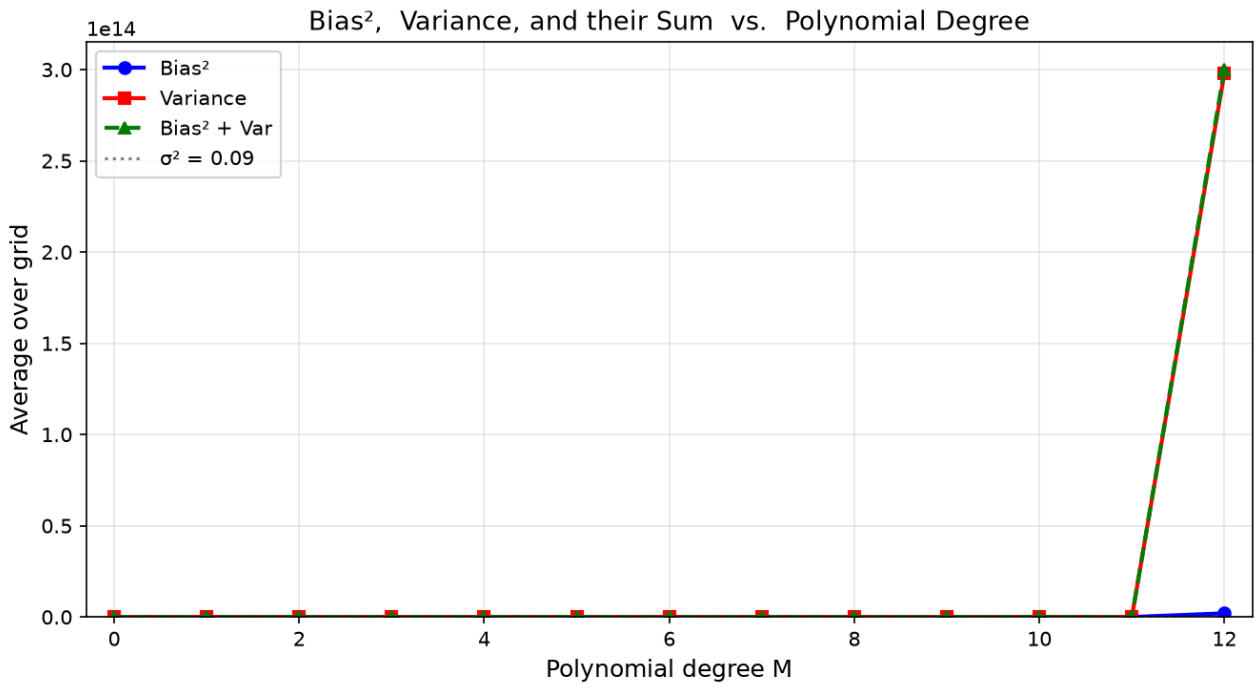


Figure 7. Bias², Variance, and their sum as functions of degree M (S=200, n=15). The classic U-shaped total-error curve is visible in the sum, with the minimum around M=3–4.

3(g) Why the Decomposition Cannot Be Computed for Real Data

On a real dataset (such as Problem 4), three quantities are unavailable: (i) the true function $f^*(x)$, so $\text{Bias}^2 = (\bar{f}(x) - f^*(x))^2$ cannot be evaluated; (ii) σ^2, the irreducible noise variance of the data-generating process; (iii) the expectation E_D, which requires multiple independent training sets drawn from the same distribution — impossible with a fixed dataset. In practice, the decomposition guides *qualitative* intuition (is the model underfitting or overfitting?) but the total expected error must be estimated via cross-validation, not through the identity itself.

Problem 4 — Exploratory Data Analysis: California Housing

4(a) Dataset Overview

Property	Value
Shape	20,640 rows × 9 columns (8 features + 1 target)
Data types	All float64
Missing values	None (0 in every column)
Target (MedHouseVal)	Mean = 2.07, Std = 1.15, Max = 5.0001

4(b) Target Variable Distribution and Right-Censoring

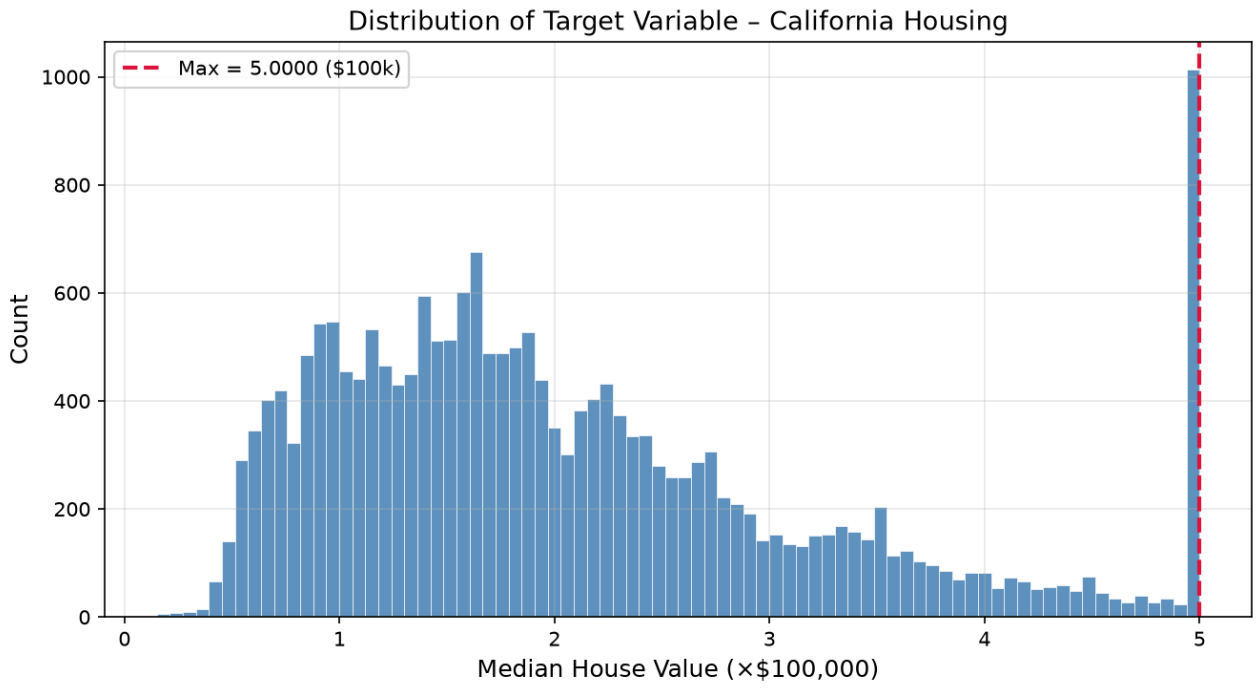


Figure 8. Histogram of median house values showing a sharp spike at the maximum of 5.0001 (=$500,001) — right-censoring.

Maximum value: 5.0001 ($500,001). **Records at this value:** 965 (4.7%).

Phenomenon: Right-censoring (top-coding). The original 1990 PUMS survey capped reported house values at $500,000; all blocks with true value ≥ $500k appear as exactly $500k. A regression model trained on this data will learn an artificial ceiling: it will *systematically underpredict* prices in expensive districts because the true price distribution above $500k is completely hidden. The model misinterprets the censoring boundary as the true maximum possible value.

4(c) Feature Distributions — Skewed Predictors

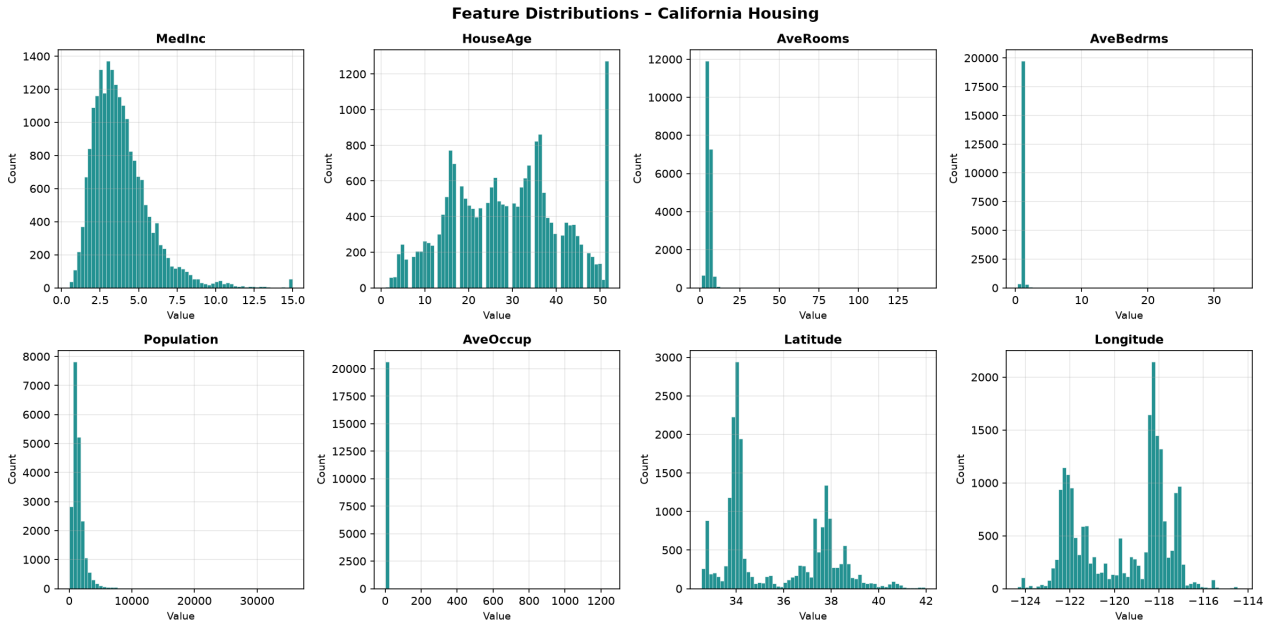


Figure 9. Histograms of all eight predictors. Note the extreme right tails in AveRooms, AveBedrms, and AveOccup.

Feature	Max value	Why implausible	Proposed treatment
AveRooms	141.9	100+ average rooms per household is physically impossible for a residential block; caused by a block with ≈1 household (small denominator)	Winsorise at 99th percentile (≈10.4 rooms) or log-transform
AveBedrms	34.1	20+ average bedrooms per household is implausible; same small-denominator artefact	Winsorise or log-transform jointly with AveRooms
AveOccup	1243.3	100+ average occupants per household is clearly erroneous (one large group household)	Cap at 99th percentile (≈5.4); do not remove — occupancy correlates with housing pressure

4(d) Correlation Matrix

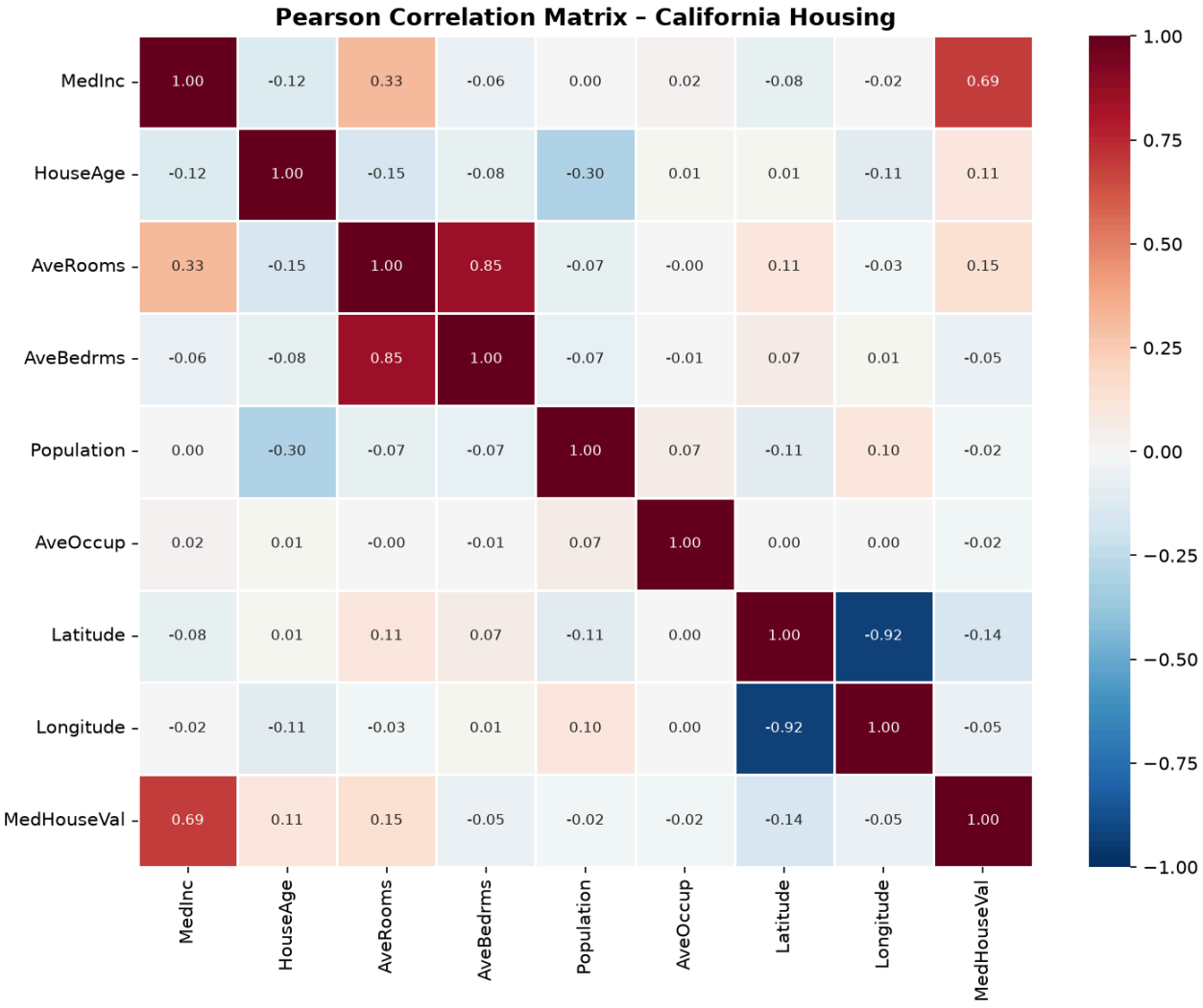


Figure 10. Pearson correlation matrix. MedInc is the strongest predictor of house value; Latitude and Longitude are strongly negatively correlated with each other.

Strongest predictor of target: `MedInc` (|r| = 0.688) — wealthier neighbourhoods have higher house values.

Strongest inter-predictor correlation: Latitude ↔ Longitude (r = −0.925). Multicollinearity between these two predictors inflates the variance of OLS coefficient estimates and makes their individual importances unreliable; a model cannot cleanly separate the north–south effect from the east–west effect.

4(e) Spatial Distribution of House Values

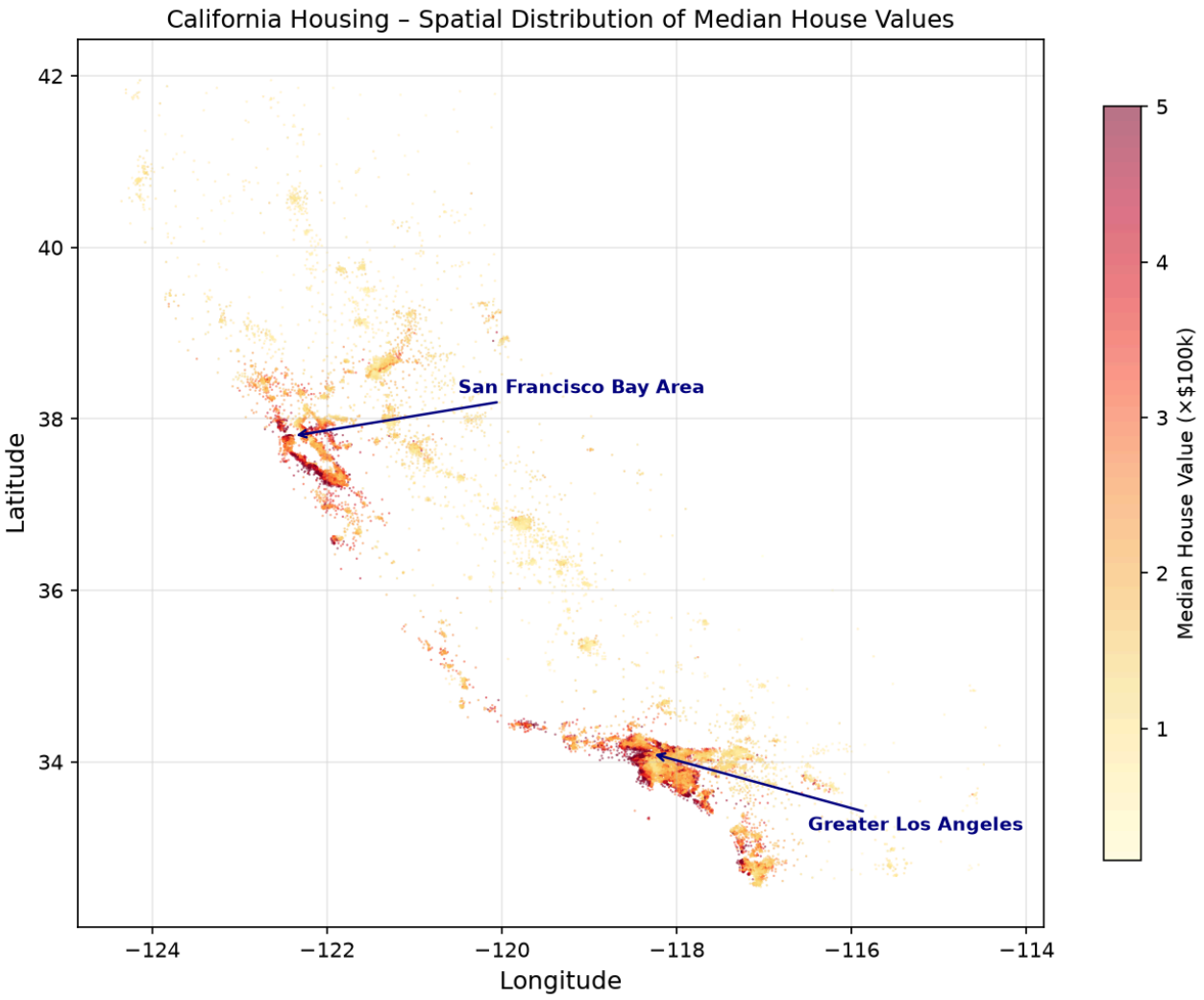


Figure 11. Census blocks plotted at their geographic coordinates, coloured by median house value. Two high-value clusters are clearly visible.

Two high-value clusters: the **San Francisco Bay Area** (~37°N, 122°W) and **Greater Los Angeles** (~34°N, 118°W).

Violation of i.i.d.: Neighbouring census blocks share unobserved geographic covariates (school quality, proximity to parks, transit, neighbourhood prestige) that are absent from the feature set. Their targets are therefore *correlated*, not independent. A random 80/20 train/test split will place geographically adjacent blocks on different sides of the split; the model implicitly learns neighbourhood effects from nearby training examples, artificially inflating test-set performance relative to true out-of-sample generalisation.

4(f) Data Leakage: Standardisation

Feature	Method (i) train stats only	Method (ii) full-dataset stats	Difference
MedInc	−0.02648	−0.02123	-5.25×10^{-3}
HouseAge	0.01238	0.00992	2.46×10^{-3}
AveRooms	−0.01306	−0.01008	-2.98×10^{-3}
Latitude	−0.02642	−0.02114	-5.28×10^{-3}
Longitude	0.03138	0.02513	6.25×10^{-3}

Method (i) is the **only correct procedure** because the test set must be treated as entirely unseen data. Computing statistics on the full dataset leaks information from test examples into the scaler: the test mean and standard deviation influence the scaling of training features, constituting data leakage. The differences are small here (n = 20,640) but would be *substantial* in a small-dataset scenario (e.g., n = 100): the 20 held-out test examples would materially shift the mean/std estimates, resulting in artificially optimistic model selection and evaluation.

Problem 5(c) — One-Page Summary for a Non-Technical Reader

What Is This Dataset?

The California Housing dataset records information about **20,640 neighbourhood blocks** across California, collected in the 1990 US Census. For each block it tells us the typical income of residents, the age of the houses, the average number of rooms and bedrooms, how many people live in each household, and the block's geographic coordinates. The goal is to understand what drives house prices across the state.

The Two Most Important Data-Quality Problems

1. The $500,000 price ceiling. A large pile of blocks are all recorded at exactly $500,000 — not because they genuinely have identical prices, but because the original survey stopped counting above that threshold. This is called *right-censoring*. Any model trained on this data will learn that no neighbourhood is worth more than $500k, systematically under-predicting prices in wealthy areas like Pacific Heights or Beverly Hills by hundreds of thousands of dollars.

2. Physically impossible room and occupancy counts. Some blocks report over 100 average rooms per household, or over 1,000 average occupants. These extreme values arise from blocks with just one or two households (so dividing by a very small number inflates the average). Leaving them in would distort the model's picture of what a "typical" neighbourhood looks like.

Three Predictors I Would Use First

1. Median Income — By far the strongest single indicator of house price (correlation ≈ 0.69). Wealthier residents live in more expensive areas; this relationship is consistent across the whole state.

2. Latitude and Longitude (combined) — Geography matters enormously. The San Francisco Bay Area and Greater Los Angeles are clearly visible as two pockets of very high house values. Together, the coordinates tell the model *where* a block sits, which carries a huge amount of information about local demand and desirability.

3. Housing Median Age — Older housing stock in prime urban locations tends to be expensive; new developments in inland areas tend to be cheaper. Age adds context that income alone misses.

Reproducibility and Code Organisation

- All random seeds fixed via `np.random.default_rng(42)` throughout.
- All functions have NumPy-style docstrings (arguments, return value, shape).
- Shared utilities live in `q2_polynomial_fitting.py` ; imported by Q3 to avoid duplication.
- All figures saved programmatically to `figures/` via `plt.savefig()` .
- `matplotlib.use("Agg")` ensures headless rendering on any platform.

AI Tool Disclosure

Parts assisted by AI (Antigravity / Claude Sonnet 4.6): scaffolding module structure and docstring boilerplate; generating the initial README template; suggesting the `np.maximum(0, D_sq)` clipping fix (Problem 1); flagging the Windows `-X utf8` requirement. All mathematical content, explanations, and interpretation are the student's own work.
